



# Resolución de Problemas Parte III (1)

Leticia Blanco

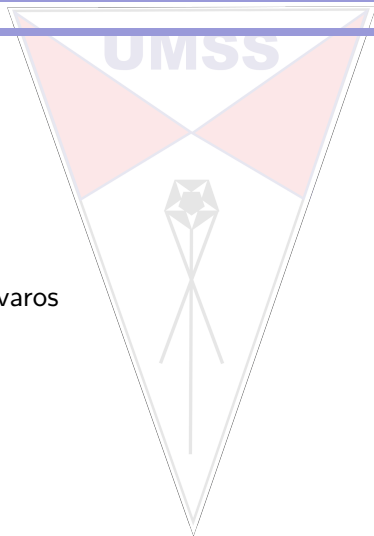
Departamento de Informática - Sistemas  
UMSS

22 de abril de 2025



# Contenido

- ▶ Algoritmos avaros

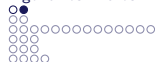




# Algoritmos avaros

## Definición

Obtiene soluciones optimas parciales bajo la suposición de que se llegará a la solución óptima total



## Algoritmos avaros

Problemas con:

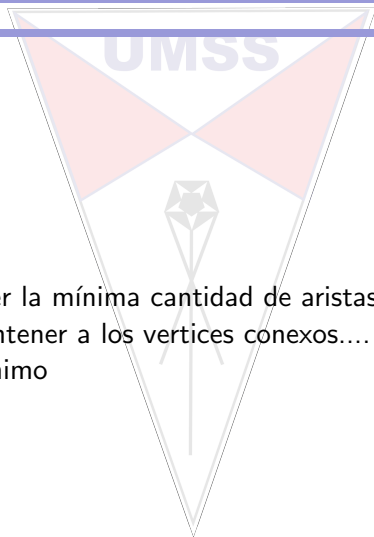
- ▶ n entradas
- ▶ salida es un subconjunto de las entradas
- ▶ funciones para
  - ▶ seleccionar un candidato - “mejor”
  - ▶ verificar si subconjunto es prometedor -- > añadir -- > solución
  - ▶ determinar si se llegó al objetivo
- ▶ maximizar/minimizar
- ▶ solución óptima



# MST

## Planteamiento

Se desea mantener la mínima cantidad de aristas y de menor peso que permitan mantener a los vertices conexos.... Árbol de recubrimiento mínimo



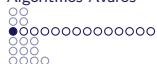


# MST

## Observaciones

- El grafo debe mantenerse conexo

*Eliminar una arista de un ciclo no desconecta el grafo*



# Kruskal

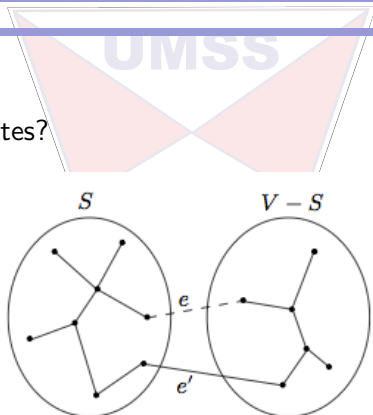
La estrategia es “armar nuevamente” el grafo, considerando aristas que no generan ciclos y las que parecen las “**mejores**” para lograr el objetivo final.

En caso de encontrar dos grafos, entonces se tiene la “propiedad de corte”



# Kruskal

Cómo unir las partes?



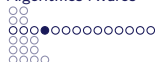
seleccionando la arista que una un vertice de una parte con un vertice de la otra, si se elije la arista menos pesada, se tiene una buena solución



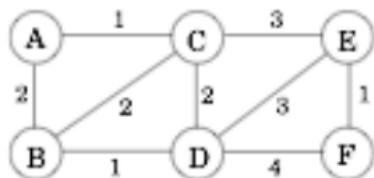


# Kruskal

```
procedure kruskal( $G, w$ )  
input :  $G = (V, E)$  adirigido,  $w$  los pesos de las aristas  
output:  $X$  que es subconjunto de aristas que representan al spanning tree  
for  $u \in V$  do  
    | makeset( $u$ )  
end  
 $X = \{\}$   
 $C = E$  aristas ordenadas por peso  
for arista  $(u, v) \in C$  do  
    | if  $find(u) \neq find(v)$  then  
        | añadir  $(u, v)$  a  $X$   
        | union( $u, v$ )  
    | end  
end
```



## Kruskal



para todo  $x \in V$ : makeSet(x)

$S = \{\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}\}$

ordenar aristas por peso

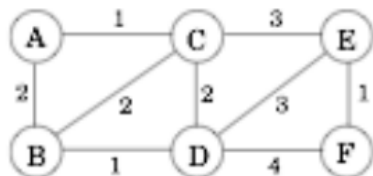
$E = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

inicializar el conjunto resultado

$X = \{\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)

$S = \{\{A\}, \{B\}, \{C\}, \{D\}, \{E\}, \{F\}\}$

$S = \{\{A, C\}, \{B\}, \{D\}, \{E\}, \{F\}\}$

ordenar aristas por peso

$E = \{(\text{A,C},1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

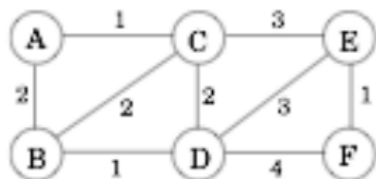
$(A,C,1) \quad \{A\} \diamond \{C\}$

inicializar el conjunto resultado

$X = \{(A,C,1)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)

$S = \{\{A, C\}, \{B\}, \{D\}, \{E\}, \{F\}\}$

$S = \{\{A, C\}, \{B, D\}, \{E\}, \{F\}\}$

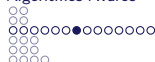
ordenar aristas por peso

$E = \{(\textcolor{red}{A}, \textcolor{red}{C}, 1), (\textcolor{red}{B}, \textcolor{red}{D}, 1), (E, F, 1), (A, B, 2),$   
 $(B, C, 2), (C, D, 2), (C, E, 3), (D, E, 3), (D, F, 4)\}$

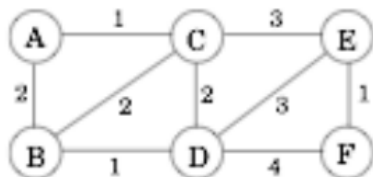
$(B, D, 1) \quad \{B\} \diamond \{D\}$

inicializar el conjunto resultado

$X = \{(A, C, 1), (B, D, 1)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)

$S = \{\{A, C\}, \{B, D\}, \{E\}, \{F\}\}$

$S = \{\{A, C\}, \{B, D\}, \{E, F\}\}$

ordenar aristas por peso

$E = \{(\text{A,C},1), (\text{B,D},1), (\text{E,F},1), (\text{A,B},2),$   
 $(\text{B,C},2), (\text{C,D},2), (\text{C,E},3), (\text{D,E},3), (\text{D,F},4)\}$

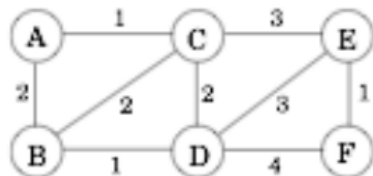
$(\text{E,F},1) \quad \{E\} \diamond \{F\}$

inicializar el conjunto resultado

$X = \{(\text{A,C},1), (\text{B,D},1), (\text{E,F},1)\}$



## Kruskal



para todo  $x \in V$ : makeSet( $x$ )

$S = \{\{A, C\}, \{B, D\}, \{E, F\}\}$

$S = \{\{A, C, B, D\}, \{E, F\}\}$

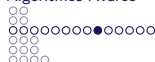
ordenar aristas por peso

$E = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

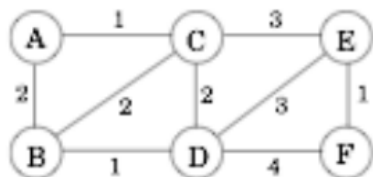
$(A,B,2) \quad \{A,C\} \diamond \{B,D\}$

inicializar el conjunto resultado

$X = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)  
 $S = \{\{A, C, B, D\}, \{E, F\}\}$

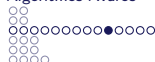
ordenar aristas por peso

$E = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

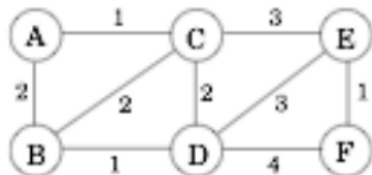
$(B,C,2) \quad \{A,C,B,D\} = \{A,C,B,D\}$

inicializar el conjunto resultado

$X = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)  
 $S = \{\{A, C, B, D\}, \{E, F\}\}$

ordenar aristas por peso

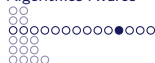
$E = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

$(C,D,2) \quad \{A,C,B,D\} = \{A,C,B,D\}$

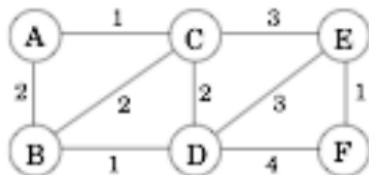
inicializar el conjunto resultado

$X = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2)\}$





## Kruskal



para todo  $x \in V$ : makeSet(x)

$S = \{\{A, C, B, D\}, \{E, F\}\}$

$S = \{\{A, C, B, D, E, F\}\}$

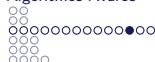
ordenar aristas por peso

$E = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2),$   
 $(B,C,2), (C,D,2), (C,E,3), (D,E,3), (D,F,4)\}$

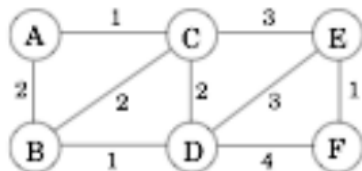
$(C,E,3) \quad \{A,C,B,D\} \diamond \{E,F\}$

inicializar el conjunto resultado

$X = \{(A,C,1), (B,D,1), (E,F,1), (A,B,2), (C,E,3)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)  
 $S = \{(A, C, B, D, E, F)\}$

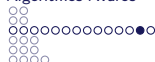
ordenar aristas por peso

$E = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2),$   
 $(B, C, 2), (C, D, 2), (C, E, 3), (D, E, 3), (D, F, 4)\}$

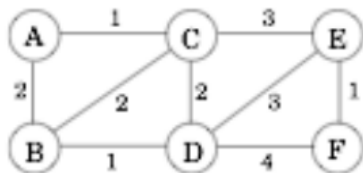
$(D, E, 3) \quad \{A, C, B, D, E, F\} = \{A, C, B, D, E, F\}$

inicializar el conjunto resultado

$X = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2), (C, E, 3)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)  
 $S = \{(A, C, B, D, E, F)\}$

ordenar aristas por peso

$E = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2),$   
 $(B, C, 2), (C, D, 2), (C, E, 3), (D, E, 3), (D, F, 4)\}$

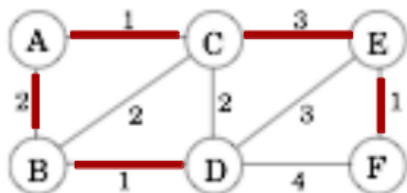
$(D, F, 4) \quad \{A, C, B, D, E, F\} = \{A, C, B, D, E, F\}$

inicializar el conjunto resultado

$X = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2), (C, E, 3)\}$



## Kruskal



para todo  $x \in V$ : makeSet(x)  
 $S = \{(A, C, B, D, E, F)\}$

ordenar aristas por peso

$E = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2),$   
 $(B, C, 2), (C, D, 2), (C, E, 3), (D, E, 3), (D, F, 4)\}$

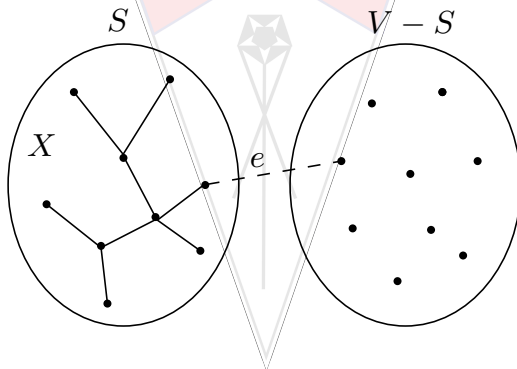
conjunto resultado

$X = \{(A, C, 1), (B, D, 1), (E, F, 1), (A, B, 2), (C, E, 3)\}$



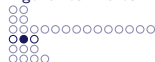
# Prim

La base es ir aumentando vertices al árbol parcial de recubrimiento



Para ello se considera los vértices a los cuales se puede llegar con

menor costo

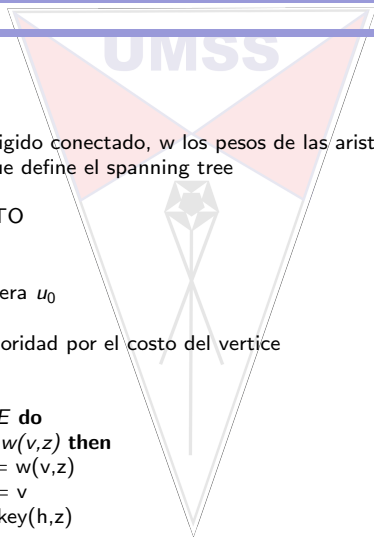


# Prim

```

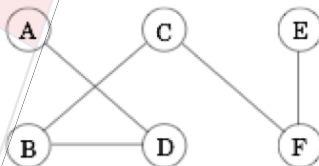
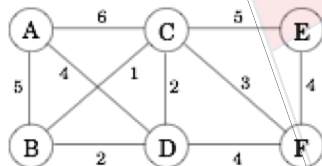
procedure prim( $G, w$ )
input :  $G = (V, E)$  adirigido conectado,  $w$  los pesos de las aristas
output: prev arreglo que define el spanning tree
for  $u \in V$  do
    |  $\text{cost}(u) = \text{INFINITO}$ 
    |  $\text{prev}(u) = \text{nil}$ 
end
Elija un vertice cualquiera  $u_0$ 
 $\text{cost}(u_0) = 0$ 
 $H = \text{makequeue}(V)$  prioridad por el costo del vertice
while  $H \neq \text{vacio}$  do
    |  $v = \text{deletemin}(H)$ 
    | for  $\text{arista } (v, z) \in E$  do
    | | if  $\text{cost}(z) > w(v, z)$  then
    | | |  $\text{cost}(z) = w(v, z)$ 
    | | |  $\text{prev}(z) = v$ 
    | | |  $\text{decreasekey}(h, z)$ 
    | | end
    | end
end

```





## Prim

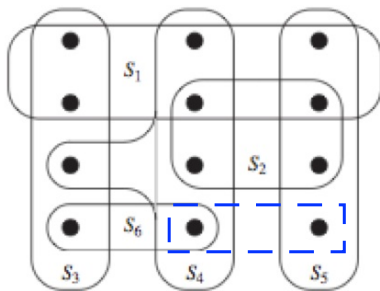


| S             | A      | B      | C      | D      | E      | F      | H             | X                                               |
|---------------|--------|--------|--------|--------|--------|--------|---------------|-------------------------------------------------|
| {}            | 0/NILL | -/NILL | -/NILL | -/NILL | -/NILL | -/NILL | {A,B,C,D,E,F} | {}                                              |
| {A}           |        | 5/A    | 6/A    | 4/A    | -/NILL | -/NILL | {D,B,C,E,F}   | {{(A,D,4)}}                                     |
| {A,D}         |        | 2/D    | 2/D    |        | -/NILL | 4/D    | {B,C,F,E}     | {{(A,D,4), (D,B,2)}}                            |
| {A,D,B}       |        |        | 1/B    |        | -/NILL | 4/D    | {C,F,E}       | {{(A,D,4), (D,B,2), (B,C,1)}}                   |
| {A,D,B,C}     |        |        |        |        | 5/C    | 3/C    | {F,E}         | {{(A,D,4), (D,B,2), (B,C,1), (C,F,3)}}          |
| {A,D,B,C,F}   |        |        |        |        | 4/F    |        | {E}           | {{(A,D,4), (D,B,2), (B,C,1), (C,F,3), (F,E,4)}} |
| {A,D,B,C,F,E} |        |        |        |        |        |        | {}            |                                                 |



## Set cover

Dado un conjunto de vértices  $X$  y un conjunto de subconjuntos  $\mathcal{F}$ , se pide encontrar el mínimo  $\mathcal{C} \subseteq \mathcal{F}$  / los vértices de  $\mathcal{C}$  contengan a todos los vértices de  $X$



$$X = 12$$

$$\mathcal{F} = 6$$

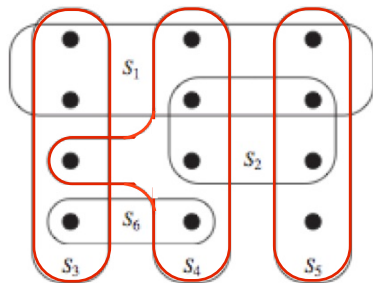
$S_1, S_2, S_3, S_4, S_5, S_6$





# Set cover

El resultado en nuestro ejemplo es que  $\mathcal{C}$  tiene 3 de los subconjuntos de  $\mathcal{F}$ , y son:



$s_1, s_4, s_2, s_3, s_5, s_6$

$s_4, s_2, s_3, s_5, s_6$

$s_2, s_3, s_5, s_6$

$s_3, s_5, s_6$

$s_1 \text{ inter } s_4, s_3, s_5, s_2, s_6 = s_1 \times$

$s_1 \text{ inter } s_2, s_3, s_5, s_6 = s_4 \checkmark$

$s_2 \text{ inter } s_3, s_5, s_6. \quad s_4 = s_2 \times$

$s_3 \text{ inter } s_5, s_6, s_4. \quad \neq s_3 \checkmark$

$s_5 \text{ inter } s_6, s_4, s_3 \neq s_5 \checkmark$

$s_6 \text{ inter } s_4, s_3, s_5 = s_6 \times$

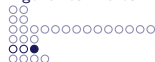
$$X = 12$$

$$\mathcal{F} = 6$$

$S_1, S_2, S_3, S_4, S_5, S_6$

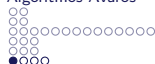
$$\mathcal{C} = 3$$

$S_3, S_4, S_5$



## Set cover

- ▶ Ordenar los conjuntos por tamaño de mayor a menor, en una cola
- ▶ Eliminar conjunto de la cola en  $S_x$
- ▶ Intersecar  $S_x$  con la union de los demás conjuntos    **U conjunto res**
- ▶ Si el resultado es  $S_x$ , este  $S_x$  se deshecha
- ▶ Si el resultado es diferente a  $S_x$ , este conjunto es parte de la solución
- ▶ continuar con el segundo paso, hasta que ya no haya conjuntos en la cola

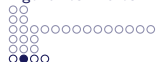


## Código de Huffman

Se quiere codificar símbolos en código binario de tal manera que puedan ser reconocidos de forma unívoca, aún cuando se juntan

### Ejemplo

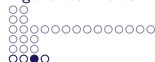
Para comprimir archivos, se puede usar el código Huffman, para ello si se cuenta con información adicional de la cantidad de ocurrencias de los símbolos, se puede tomar algunas decisiones.



## Código de Huffman

Si por ejemplo el archivo tiene 4 símbolos: A, B, C y D, y su frecuencia de aparición de estos símbolos en el archivo son:

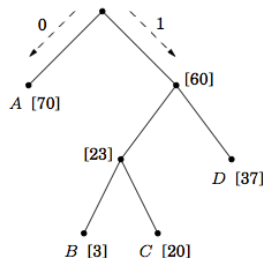
| Symbol   | Frequency  |
|----------|------------|
| <i>A</i> | 70 million |
| <i>B</i> | 3 million  |
| <i>C</i> | 20 million |
| <i>D</i> | 37 million |



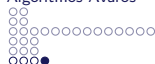
## Código de Huffman

Entonces, es bueno que el símbolo con mayor frecuencia ocupe menos símbolos del código, luego D y así sucesivamente de acuerdo a la frecuencia. Una buena opción es:

| Symbol | Codeword |
|--------|----------|
| A      | 0        |
| B      | 100      |
| C      | 101      |
| D      | 11       |



V



# Código de Huffman

El algoritmo es:

procedure Huffman( $f$ )

Input: An array  $f[1 \dots n]$  of frequencies

Output: An encoding tree with  $n$  leaves

let  $H$  be a priority queue of integers, ordered by  $f$

for  $i=1$  to  $n$ : insert( $H, i$ )

for  $k=n+1$  to  $2n-1$ :

$i = \text{deletemin}(H)$ ,  $j = \text{deletemin}(H)$

    create a node numbered  $k$  with children  $i, j$

$f[k] = f[i] + f[j]$

    insert( $H, k$ )